

Policy Gradient Preliminaries

The Stochastic Gradient Estimator and How Every Algorithm Varies It

Reinforcement Learning

Sai Sampath Kedari

sampath@umich.edu

Contents

1	Purpose	2
2	Three Objectives, All Deterministic	2
3	Why We Cannot Compute the Objective or Its Gradient	3
4	From Exact Ascent to Stochastic Approximation	3
5	Three Levels: Problem, Estimator, Recursion	3
6	The Universal Estimator Notation	4
7	The Decomposition That Organizes the Series	5
8	The Game: What All Policy Gradient Algorithms Are Doing	5
9	Notation and Standing Assumptions	6

1 Purpose

Every policy gradient algorithm in this series, REINFORCE, actor-critic, TRPO, PPO, SAC, and the rest, solves the same optimization problem with the same recursion. They differ only in *how they estimate one gradient*. This short report fixes that common ground once: the objective, its gradient, why neither can be computed exactly, the resulting stochastic-approximation recursion, and a single notation and decomposition that every later report reuses. Nothing here is derived from scratch; the gradients are quoted from the policy gradient theorem reports. The one contribution is the estimator decomposition of Section 7, which is the frame through which every algorithm is read.

2 Three Objectives, All Deterministic

There are three standard performance objectives. In each, π_θ is a policy parameterized by $\theta \in \mathbb{R}^{d'}$, and the objective $J : \mathbb{R}^{d'} \rightarrow \mathbb{R}$ is a deterministic scalar function of θ .

1. Infinite-horizon discounted. Weight each start state by ρ_0 and score it by its value:

$$J(\theta) = \sum_{s_0} \rho_0(s_0) V^{\pi_\theta}(s_0) = \mathbb{E}_{S_0 \sim \rho_0} [V^{\pi_\theta}(S_0)].$$

Its gradient (policy gradient theorem, discounted route):

$$\nabla_\theta J(\theta) = \frac{1}{1-\gamma} \mathbb{E}_{S \sim d^{\pi_\theta, \rho_0}, A \sim \pi_\theta(\cdot|S)} [Q^{\pi_\theta}(S, A) \nabla_\theta \log \pi_\theta(A | S)].$$

2. Continual average reward. The long-run reward rate under the stationary distribution μ_{π_θ} :

$$J(\theta) = r(\pi_\theta) = \sum_s \mu_{\pi_\theta}(s) \sum_a \pi_\theta(a | s) r(s, a) = \mathbb{E}_{S \sim \mu_{\pi_\theta}, A \sim \pi_\theta(\cdot|S)} [r(S, A)].$$

Its gradient (average-reward policy gradient theorem):

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \mu_{\pi_\theta}, A \sim \pi_\theta(\cdot|S)} [Q^{\pi_\theta}(S, A) \nabla_\theta \log \pi_\theta(A | S)].$$

3. Episodic, discretized by trajectories. When episodes terminate, write the objective directly over trajectories $\tau \sim p_\theta$ with return G_0 :

$$J(\theta) = \sum_\tau p_\theta(\tau) G_0 = \mathbb{E}_{\tau \sim p_\theta} [G_0].$$

Its gradient (episodic trajectory route, reward-to-go form):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T-1} \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t | S_t) \right].$$

The three objectives share one shape: an expectation of a reward-like quantity under a distribution induced by π_θ , and each gradient is an expectation of the score $\nabla_\theta \log \pi_\theta$ weighted by a value (Q^{π_θ} or the return). This shared shape is exactly what lets one notation and one analysis serve all three.

3 Why We Cannot Compute the Objective or Its Gradient

Although $J(\theta)$ and $\nabla_{\theta}J(\theta)$ are deterministic functions of θ , neither has a usable closed form.

- **Unknown dynamics.** The expectations run over the transition kernel $p(s' | s, a)$, which the agent does not know. The objective cannot be evaluated from the definition.
- **Intractable even if known.** Even with p in hand, the expectation is over entire trajectories: the state and action spaces may be large or continuous, and the horizon long, so the sum or integral has no closed form we can compute.

The same holds for the gradient $\nabla_{\theta}J(\theta) : \mathbb{R}^{d'} \rightarrow \mathbb{R}^{d'}$: it is an expectation over the induced distribution, exactly computable nowhere. This is the central obstruction, and every algorithm in the series is a response to it.

4 From Exact Ascent to Stochastic Approximation

The goal, in all three cases, is the optimization problem

$$\theta^* \in \arg \max_{\theta \in \Theta} J(\theta).$$

The textbook way to solve a smooth maximization is gradient ascent: from the current iterate θ_k , step along the gradient with step size α_k ,

$$\theta_{k+1} = \theta_k + \alpha_k \nabla_{\theta}J(\theta_k) \quad (\text{exact, deterministic ascent}).$$

To see what this recursion actually asks for, write it out with the objective and its gradient in place. Take the infinite-horizon discounted case: the problem is

$$\theta^* \in \arg \max_{\theta \in \Theta} J(\theta), \quad J(\theta) = \mathbb{E}_{S_0 \sim \rho_0} [V^{\pi_{\theta}}(S_0)],$$

and the exact ascent recursion is

$$\theta_{k+1} = \theta_k + \alpha_k \nabla_{\theta}J(\theta_k) = \theta_k + \alpha_k \frac{1}{1 - \gamma} \mathbb{E}_{S \sim d^{\pi_{\theta_k}, \rho_0}, A \sim \pi_{\theta_k}(\cdot | S)} [Q^{\pi_{\theta_k}}(S, A) \nabla_{\theta} \log \pi_{\theta_k}(A | S)].$$

We cannot run this recursion. The update direction is an expectation over the visitation $d^{\pi_{\theta_k}, \rho_0}$ and the policy, weighted by the true value $Q^{\pi_{\theta_k}}$, and none of these is available in closed form (previous section). The single move that defines this entire subject is to replace the exact gradient $\nabla_{\theta}J(\theta_k)$ by a *sampled estimator* $\hat{g}_k$ formed from data the agent collects by running the current policy. The deterministic recursion becomes a *stochastic-approximation* recursion, and the study of policy gradient algorithms is the study of how to build that estimator well.

5 Three Levels: Problem, Estimator, Recursion

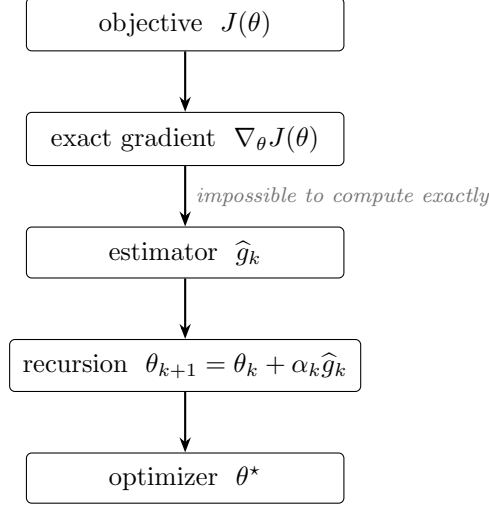
The subject separates cleanly into three levels, and keeping them apart is the backbone of every later report.

Key Result

$\underbrace{J(\theta)}$	$\underbrace{\hat{g}_k \longrightarrow \nabla_{\theta}J(\theta_k)}$	$\underbrace{\theta_{k+1} = \theta_k + \alpha_k \hat{g}_k \longrightarrow \theta^*}$
optimization problem	estimator: local gradient	recursion: reaches the optimizer

The estimator $\hat{g}_k$ does *not* estimate θ^* . It estimates only the local gradient $\nabla_{\theta}J(\theta_k)$; the stochastic-approximation recursion is what turns a stream of local estimates into progress toward θ^* . So a policy gradient method is a *statistical estimation problem* (build $\hat{g}_k$) wrapped inside a *stochastic-approximation algorithm* (iterate to θ^*), and the two are analyzed separately.

The whole pipeline, from the object we want to the point we reach:



Two boundary facts. If the estimator were exact, $\hat{g}_k = g_k$, the recursion collapses to ordinary deterministic gradient ascent: stochastic approximation contains exact ascent as its noise-free special case. And unlike classical statistical inference, where all data come from one fixed distribution P_{θ^*} , here the data come from the current policy π_{θ_k} , so the sampling distribution $\mathcal{D}_{\theta_k}$ moves after every update. That moving distribution is the defining difference between this setting and ordinary estimation.

6 The Universal Estimator Notation

We fix one notation for the whole series. Let the exact gradient field and its value at the current iterate be

$$g(\theta) := \nabla_{\theta}J(\theta), \quad g_k := g(\theta_k) = \nabla_{\theta}J(\theta_k) \in \mathbb{R}^{d'}.$$

At iteration k we collect random data by running the current policy π_{θ_k} ,

$$\mathcal{B}_k \sim \mathcal{D}_{\theta_k},$$

where $\mathcal{B}_k$ is whatever the algorithm samples. Its form varies across the series:

$$\mathcal{B}_k = \begin{cases} \tau_k = (S_0, A_0, R_1, \dots, S_T) & \text{full trajectory} & (\text{REINFORCE}) \\ (S_t, A_t, R_{t+1}, S_{t+1}) & \text{single transition} & (\text{TD / actor-critic}) \\ \{(s_i, a_i, r_i, s'_i)\}_{i=1}^B & \text{minibatch / replay batch} & (\text{off-policy}) \\ \{\tau_k^{(i)}\}_{i=1}^G & \text{group of rollouts} & (\text{GRPO}) \end{cases}$$

From this data we form the *stochastic gradient estimator*

$$\hat{g}_k := \hat{g}(\theta_k; \mathcal{B}_k) \in \mathbb{R}^{d'},$$

and take the stochastic-approximation step

Key Result

$$\theta_{k+1} = \theta_k + \alpha_k \hat{g}_k, \quad g_k = \nabla_{\theta} J(\theta_k), \quad \mathcal{B}_k \sim \mathcal{D}_{\theta_k}, \quad \hat{g}_k = \hat{g}(\theta_k; \mathcal{B}_k).$$

7 The Decomposition That Organizes the Series

Every estimator in the series is read through one decomposition. Split $\hat{g}_k$ into the true gradient, a bias, and zero-mean noise:

Key Result

$$\hat{g}_k = \underbrace{g_k}_{\text{true gradient}} + \underbrace{b_k}_{\text{bias}} + \underbrace{\varepsilon_k}_{\text{zero-mean noise}}, \quad \mathbb{E}[\varepsilon_k \mid \theta_k] = 0,$$

so that

$$\mathbb{E}[\hat{g}_k \mid \theta_k] = g_k + b_k.$$

The estimator is *unbiased* when $b_k = 0$ (then $\mathbb{E}[\hat{g}_k \mid \theta_k] = g_k$) and *biased* when $b_k \neq 0$. Both g_k and b_k are deterministic vectors; only ε_k is random. We do not know b_k in general: this is an analytical decomposition that names the two error modes, not a quantity we compute (if we knew b_k we would subtract it and be unbiased).

Unbiased ($b_k = 0$)	Biased ($b_k \neq 0$)
REINFORCE	actor-critic (imperfect critic)
REINFORCE with baseline	GAE with $\lambda < 1$
	PPO clipping, TD learning
	off-policy corrections

Reading any algorithm then reduces to two questions: what is its bias b_k , and what is the variance $\text{Var}(\varepsilon_k \mid \theta_k)$ of its noise. (The decomposition presumes a score-function estimator, the $\nabla_{\theta} \log \pi_{\theta} \cdot Q$ form. Deterministic and reparameterized policies use a different form of $\hat{g}_k$ entirely, the pathwise gradient of Axis 5 below; the same bias-variance reading applies once that form is in hand.)

8 The Game: What All Policy Gradient Algorithms Are Doing

Solving $\theta^* = \arg \max_{\theta} J(\theta)$ efficiently means building a better $\hat{g}_k$, or feeding it into a better update. Almost every algorithm in the series is a move along one of a few axes.

Axis 1: Bias. Keep the estimator unbiased ($b_k = 0$) so the recursion aims at the true gradient. When you cannot, or when it pays, accept a controlled bias ($b_k \neq 0$) in exchange for something else, usually a large cut in variance (bootstrapping is the standard way to do this).

Axis 2: Variance. Given the bias you have chosen, shrink $\text{Var}(\varepsilon_k \mid \theta_k)$. Lower noise means faster, steadier convergence of the same recursion. Control variates (baselines) are the first tool and cost no bias.

Axis 3: Geometry. Change how $\hat{g}_k$ enters the update. Instead of the plain Euclidean step, precondition by the Fisher information (natural gradient) or constrain the step in KL (trust region). This leaves the estimator alone and improves the *dynamics* of the recursion.

Axis 4: Form of the estimator. Everything above uses the *score-function* estimator $\nabla_{\theta} \log \pi_{\theta} \cdot Q$. A different construction differentiates *through* the value by the chain rule, the *pathwise* (reparameterized) gradient, used by deterministic and reparameterized policies. This is a different object, not a different weight or metric, so it is analyzed on its own terms.

Independent knobs. *Bias, variance, geometry, and estimator form are largely independent, which is why real algorithms combine them: one can bootstrap (bias) with a baseline (variance) under a trust region (geometry). Each later report states which axes it turns. Three further directions sit slightly apart and are treated where they arise: correcting for off-policy data (importance sampling and its variants), regularizing the objective with entropy (the maximum-entropy corner, where policy gradient meets inference), and parallelizing data collection (which changes the noise structure rather than the estimator).*

Concretely, the series walks the following path. Each report changes exactly one ingredient of $\hat{g}_k$ or of the update, which is what makes the sequence feel inevitable rather than arbitrary.

Report	What it changes
REINFORCE	the estimator (sampled return, $b_k = 0$)
REINFORCE with baseline	variance (control variate)
Actor-critic	bootstrapping (buys variance with bias)
GAE	the bias-variance knob (λ)
NPG	geometry (Fisher metric)
TRPO	geometry (KL trust region)
PPO	geometry (approximate trust region)
DDPG / TD3	pathwise estimator (and critic stability)
SAC	the objective (entropy, max-entropy corner)

9 Notation and Standing Assumptions

Fixed once here and not redefined in later reports.

Symbol	Meaning
$\theta \in \mathbb{R}^{d'}$	policy parameters
$\pi_\theta(a \mid s)$	parameterized policy
$J(\theta)$	performance objective, deterministic $\mathbb{R}^{d'} \rightarrow \mathbb{R}$
$g(\theta) = \nabla_\theta J(\theta)$	exact gradient field; $g_k = g(\theta_k)$
ρ_0	start-state distribution
d^{π_θ, ρ_0}	discounted state-visitation distribution
μ_{π_θ}	stationary distribution (average-reward case)
$V^{\pi_\theta}, Q^{\pi_\theta}$	state- and action-value functions under π_θ
$A^{\pi_\theta}(s, a) = Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s)$	advantage
$\gamma \in [0, 1)$	discount factor (discounted case)
G_t	return from step t
$z_\theta(s, a) = \nabla_\theta \log \pi_\theta(a \mid s)$	score function
$\mathcal{B}_k \sim \mathcal{D}_{\theta_k}$	random data collected at iteration k
$\hat{g}_k = \hat{g}(\theta_k; \mathcal{B}_k)$	stochastic gradient estimator
$\hat{g}_k = g_k + b_k + \varepsilon_k$	bias-noise decomposition
α_k	step size

Standing assumptions. Unless a report states otherwise: a Markov decision process with the stated state and action spaces; a policy π_θ differentiable in θ with a well-defined score; bounded rewards; and, in the discounted case, $\gamma < 1$. Step sizes, where convergence is discussed, satisfy the usual Robbins-Monro conditions $\sum_k \alpha_k = \infty$ and $\sum_k \alpha_k^2 < \infty$.